Kristianstad University
SE-291 88 Kristianstad
+46 44-250 30 00
www.hkr.se

Lab Report, Fullstack Development (DA219B)

Spring Semester 2026

Fullstack Development

Lab 1 Report

Liam Jörgensen

Content

1. System Overview

The app targets people who workout and want to track their gym sessions. The application provides the ability to create workouts, and log sessions, and limited statistics. When creating a workout, a name for it, a description, tags and exercises can be added. The user can then log the workout as a session, add sets to the exercises, and specify reps and weight. Furthermore, each user has a profile where they can set a profile picture, write a short bio, and view some general statistics. The profile of a user can be viewed by any user. The dashboard provides the weekly volume and the latest completed session

2. ERD

Users Collection:

- Used to store user information.
- The id of users are references by workouts and sessions, linking a session/workout to a user.

Sessions Collection:

- Used to store information about sessions users have logged.
- Uses userId to link the session to a user and a workoutId to link a workout to the session

Workouts Collection:

- Used to store information about a workout a user has created.
- Uses userId to link a user to the workout and the id of the workout is referenced by sessions.

Exercices Collection:

- Used as a collection of exercises to choose from and add to a workout.
- It does not have any real relationships to other collections since both sessions and workouts hold snapshots of exercises.

3. Two API-Endpoints

3.1 Login route

3.1.1 Route: http://localhost:3000/login

3.1.2 Method: POST

3.1.3 Expected Request body

```
{
email: "Email@email.email",
password: "StrongPassword123!"
}
```

3.1.4 Example Response

```
{
success: true,
data:{
    "_id": "69df90d889bad7254b1f98d4",
    "fullname": "Liam Jörgensen",
    "username": "FlurryTV",
    "email": "zoonspoon123@gmail.com",
    "createdAt": "2026-04-15T13:21:28.534Z",
    "bio": "Test",
    "updatedAt": "2026-04-26T15:08:21.816Z",
    "profilePicture": ""
    }
}
```

3.2 Register

3.2.1 Route: http://localhost:3000/register

3.2.2 Method: POST

3.2.3 Expected Request body

```
{
fullname: "You Name",
username: "YourUsername12",
email: "your@email.com",
password: "SecretPassword123!"
}
```

3.2.4 Example Response

```
{
success: true,
data: "Account successfully registered!"
}
```

4. Reflection

One challenge I faced was structuring backend data fetching in a way that made the data easy to access, update, and sort efficiently. For sessions, I needed to retrieve all sessions belonging to a user and then divide them into two separate arrays: one for upcoming sessions and one for past sessions. Additionally, when a session was marked as completed, it needed to move to the past sessions array, and if it was changed back to incomplete, it needed to return to the upcoming sessions array.

Because the session data needed to remain easily accessible, mutable, and sortable across the application, I chose to manage it through a context. This provided centralized access and simplified updates. I then used a `useMemo` hook with the sessions array as a dependency, allowing the data to be efficiently recalculated only when sessions changed.

This approach made it possible to maintain two dynamically updated arrays based on sorting logic, while ensuring the data remained performant, consistent, and easy to work with throughout the application.

I had no prior experience with `useMemo`, but I found it to be really useful and could be something good to know for future react projects.

5. Feature iteration:

One feature that shows iteration during development is the session management system.

In commit **832b7f6**, the initial implementation introduced the basic ability to create workout sessions with a selected workout and a date. This established the base functionality for scheduling, storing sessions and showing sessions.

In commit **751c11c**, the feature was improved by adding completion toggling and automatic organization into past and upcoming sessions. This refinement expanded the original implementation by introducing state-based behavior and automatically sorting sessions in a predictable manner

6. Extra work and Libraries used

Additional tools were used to improve both development and functionality. TypeScript added type safety and maintainability, TailwindCSS responsive styling, and React Router DOM enabled frontend navigation. JWT and bcrypt provided secure authentication, while Morgan supported request logging for debugging. Nodemon improved the development workflow by enabling automatic restarts.